o00range
https://github.com/n0-space

# BITCOIN-ADJACENT COORDINATION AS A WEIGHTED PRE-SETTLEMENT MARKET: WEAKWORK, BRAIDS, AND MINING IMPLICATIONS

ODA BLORANGE[1]

Abstract. We develop a stricter theorem-lemma style treatment of a Bitcoin-adjacent overlay in which coordination objects compete in a pre-settlement market before optional anchoring into Bitcoin blockspace. Starting from the distinction between consensus-valid state and non-consensus staging state, we formalize weakwork as a locally verifiable proof-of-effort object, braids as weighted partially ordered candidate histories, and knots as checkpoint selections over braid space. We prove elementary invariants about deterministic replay, monotonicity of additive weight functions, admissibility under bounded execution, and the compressibility of expressive histories into settlement commitments.

We then study mining-market implications. In particular, we show that once overlay objects have both relay-side and settlement-side value, a two-stage pricing problem emerges: first for attention and inclusion inside the overlay, and second for final anchoring into Bitcoin blockspace. This creates a new economic interface between hash markets, relay markets, and execution markets without modifying Bitcoin consensus. The resulting framework clarifies how a Bitcoin-rooted coordination network may remain subordinate to Bitcoin's final settlement while expanding the priced surface of digital action.

## Contents

1. Introduction 2
2. Base Objects and Separation of Layers 3
3. Weakwork 3
3.1. Definition and admissibility 3
3.2. Economic meaning 4
4. Dependency Graphs and Braids 5
4.1. Partial order 5
4.2. Weight functions 5
5. Replay and Overlay State 6
5.1. Bounded execution 6
5.2. Knots 7
6. Compression Into Bitcoin Settlement 7
6.1. Anchors 7
7. Two-Stage Markets 8
7.1. Relay-side market 8
7.2. Settlement-side market 8

8. Mining-Market Implications 9
8.1. Weakwork and hash allocation 9
8.2. Anchor-driven miner incentives 9
8.3. Braid markets and mempool analogues 10
9. Adversarial Considerations 10
10. Client-Type Implications 11
11. Main Synthesis 11
12. Conclusion 12
13. The Order of Agents in Kapnet 12
13.1. Preliminaries 12
13.2. Agent classes 13
13.3. The productive order 14
13.4. Flow maps 15
13.5. Treasury and continuity 15
14. Economy of Units 16
14.1. Overview 16
14.2. Bitcoin-native units 17
14.3. Physical production units 17
14.4. Weakwork units 18
14.5. Hedlbits and postage 19
14.6. Satdepth and adjusted satdepth 19
14.7. Hodlyield 20
14.8. Cross-unit pricing identities 20
14.9. Unit conversion and market surfaces 21
14.10. A concise unit summary 21
14.11. Interpretive conclusion 22
References 22

## 1. Introduction

The base Bitcoin network prices final settlement. It does not natively price every intermediate stage of coordination preceding settlement. A user may deliberate, negotiate, signal, rank, validate, batch, compress, and only then settle. The present paper studies a mathematical overlay in which these pre-settlement acts become first-class competing objects.

The central thesis is simple. Bitcoin's mempool already demonstrates that *visibility* and *finality* are not identical. A transaction can be known before it is mined. Generalizing this observation, one may construct a coordination system in which typed messages are admitted, ranked, executed, and checkpointed before any subset is committed to Bitcoin. If the ranking resource is scarce and costly, then a market arises upstream of blockspace. We model that upstream market via weakwork and braid competition.

## 2. Base Objects and Separation of Layers

**Definition 2.1** (Bitcoin-valid settlement space)**.** Let $\mathcal{B}$ denote the set of Bitcoin-valid settlement artifacts, namely valid transactions, transaction packages, blocks, and chain extensions under base consensus rules $C_B$.

**Definition 2.2** (Overlay message space)**.** Let $\mathcal{M}$ denote the set of overlay messages. Each $m \in \mathcal{M}$ is a tuple

$$m = (\iota(m), p(m), d(m), a(m), \kappa(m))$$

where $\iota(m)$ is its identity commitment, $p(m)$ its payload, $d(m)$ its dependency data, $a(m)$ its admission bundle, and $\kappa(m)$ its optional Bitcoin anchoring instruction.

**Definition 2.3** (Consensus-separation map)**.** A *consensus-separation map* is a partial map

$$\pi_{\mathrm{BTC}} : \mathcal{M} \rightharpoonup \mathcal{B}$$

which is undefined for most overlay messages and defined only for those messages, batches, or checkpoints that are to be anchored into Bitcoin.

**Axiom 2.4** (Layer separation)**.** For all $m \in \mathcal{M}$, the existence, propagation, ordering, execution, or rejection of $m$ inside the overlay does not alter the Bitcoin validity predicate on $\mathcal{B}$.

*Remark* 2.5. This is the foundational constraint. The overlay may be economically important, computationally rich, and socially consequential, yet remains external to base Bitcoin validity unless and until it is expressed through a Bitcoin-valid settlement artifact.

## 3. Weakwork

### 3.1. Definition and admissibility.

**Definition 3.1** (Weakwork witness)**.** A *weakwork witness* is a tuple

$$\omega = (x, \nu, \delta)$$

where $x$ is a committed message digest, $\nu$ is a nonce or nonce schedule, and $\delta$ is auxiliary metadata sufficient to verify that $x$ satisfies a publicly specified subtarget condition

$$H(x, \nu, \delta) \leq T_w$$

for some overlay target $T_w$.

**Definition 3.2** (Weakwork score)**.** Given a valid weakwork witness $\omega$, define its score

$$s_w(\omega) \in \mathbb{R}_{\geq 0}$$

as any monotone decreasing function of the realized hash value relative to target, for example

$$s_w(\omega) = \log\left(\frac{T_w}{H(x, \nu, \delta)}\right)$$

when the expression is well-defined.

**Definition 3.3** (Message-admissible weakwork)**.** A message $m$ is *weakwork-admissible* if there exists a weakwork witness $\omega_m$ bound to $\iota(m)$ such that the overlay's local verification predicate

$$V_w(m, \omega_m) = 1$$

holds.

**Lemma 3.4** (Local verifiability)**.** *For fixed weakwork parameters and fixed hash function $H$, admissibility of $\omega_m$ is decidable in deterministic polynomial time in the size of the witness.*

*Proof.* The verifier need only recompute the bound commitment between $\omega_m$ and $\iota(m)$, evaluate $H$ on the prescribed input, and compare the result to $T_w$. All such operations are finite and deterministic in the witness size. □

**Lemma 3.5** (Binding)**.** *Suppose the message identity commitment $\iota(m)$ is collision resistant in the relevant security model. Then a weakwork witness valid for $m$ cannot be transferred to a distinct message $m' \neq m$ without either finding a collision in $\iota$ or recomputing a valid witness bound to $m'$.*

*Proof.* If a witness for $m$ is accepted for $m'$, then either the verification predicate ignores the committed identity, contradicting the construction, or the identities match in a way that violates collision resistance, or a new valid witness was constructed. These are the only possibilities. □

*Remark* 3.6. Weakwork is not base consensus proof-of-work. It is a local proof-of-effort market object. Its function is not to determine the canonical Bitcoin chain, but to price the right to consume overlay relay, ranking, or execution attention.

### 3.2. **Economic meaning.**

**Definition 3.7** (Marginal weakwork cost)**.** Let $c_w(\omega)$ denote the expected opportunity cost of producing a weakwork witness $\omega$ under prevailing hardware, energy, and market conditions.

**Definition 3.8** (Weakwork density)**.** For a message $m$ with serialized size $\ell(m)$ and valid witness $\omega_m$, define

$$D_w(m) = \frac{s_w(\omega_m)}{\ell(m)}.$$

**Proposition 3.9** (Spam resistance by costly ranking)**.** *If relay policy prioritizes messages by a nondecreasing function of weakwork density and weakwork production has positive marginal cost, then flooding the relay layer with low-value messages incurs a strictly increasing resource burden on the sender.*

*Proof.* Under the policy assumption, a sender seeking to preserve visibility under congestion must either increase score or shrink message size. If score requires positive expected cost, then maintaining rank across many messages raises total cost. □

## 4. Dependency Graphs and Braids

### 4.1. Partial order.

**Definition 4.1** (Dependency relation)**.** Let $\mathcal{S} \subseteq \mathcal{M}$ be a set of admitted messages. A binary relation

$$\prec \subseteq \mathcal{S} \times \mathcal{S}$$

is called a *dependency relation* if $m_i \prec m_j$ means that $m_j$ depends on the prior availability, validity, or interpretive existence of $m_i$.

**Definition 4.2** (Admissible message graph)**.** An admissible message graph is a directed graph

$$G = (V, E)$$

with $V \subseteq \mathcal{S}$ and $E \subseteq V \times V$ such that $(u, v) \in E$ implies $u \prec v$.

**Definition 4.3** (Acyclic admissibility)**.** A message graph is *acyclically admissible* if it contains no directed cycles.

**Lemma 4.4** (Topological replay order)**.** *Every finite acyclically admissible message graph admits at least one topological ordering.*

*Proof.* This is the standard theorem for finite directed acyclic graphs. □

**Definition 4.5** (Braid)**.** A *braid* is a finite acyclically admissible weighted message graph

$$\beta = (V_\beta, E_\beta, w_\beta)$$

where

$$w_\beta : V_\beta \to \mathbb{R}_{\geq 0}$$

assigns a nonnegative weight to each message.

**Definition 4.6** (Braid weight)**.** The total weight of a braid is

$$W(\beta) = \sum_{m \in V_\beta} w_\beta(m).$$

### 4.2. Weight functions.

**Definition 4.7** (Additive weight model)**.** A braid weight model is *additive* if

$$w_\beta(m) = \alpha_w s_w(\omega_m) + \alpha_f f_m + \alpha_b b_m + \alpha_v v_m$$

for nonnegative coefficients $\alpha_w, \alpha_f, \alpha_b, \alpha_v$, where $s_w$ is weakwork score, $f_m$ a fee component, $b_m$ a burn component, and $v_m$ a validation or reputation component.

**Lemma 4.8** (Monotonicity)**.** *If all coefficients in the additive weight model are nonnegative, then for any braid $\beta$ and any admitted message $m \notin V_\beta$ with $w_\beta(m) \geq 0$, the extended braid*

$$\beta' = \beta \cup \{m\}$$

*satisfies*

$$W(\beta') \geq W(\beta).$$

*Proof.* Immediate from additivity and nonnegativity. □

**Corollary 4.9** (Strict improvement)**.** *If $w_\beta(m) > 0$, then*

$$W(\beta') > W(\beta).$$

*Remark* 4.10. Monotonicity alone does not determine braid selection, because dependency conflicts, replay bounds, namespace rules, and anti-double-expression constraints can make a locally heavier extension globally inadmissible.

## 5. Replay and Overlay State

### 5.1. Bounded execution.

**Definition 5.1** (State root set)**.** Let $\mathcal{R}$ be the set of valid overlay state roots. A replay engine is a partial function

$$\mathsf{Exec} : \mathcal{R} \times \mathcal{M} \rightharpoonup \mathcal{R} \times \mathcal{T} \times \mathbb{N}$$

where $\mathcal{T}$ is the set of execution trace commitments and the third output is resource cost.

**Axiom 5.2** (Bounded halting)**.** There exists $L \in \mathbb{N}$ such that for any admissible input pair $(\rho, m)$, the execution

$$\mathsf{Exec}(\rho, m)$$

either halts within at most $L$ steps with deterministic output or fails in a deterministic and non-ambiguous manner.

**Definition 5.3** (Replayability)**.** A braid $\beta$ is *replayable from root* $\rho_0$ if some topological ordering $(m_1, \ldots, m_n)$ of $V_\beta$ satisfies

$$(\rho_i, \chi_i, c_i) = \mathsf{Exec}(\rho_{i-1}, m_i)$$

for all $i = 1, \ldots, n$ without failure.

**Lemma 5.4** (Determinism of replay outcome)**.** *Suppose* $\mathsf{Exec}$ *is deterministic and all message semantics are path-independent under admissible topological orderings. Then any two topological orderings of a replayable braid yield the same terminal state root.*

*Proof.* Let $(m_1, \ldots, m_n)$ and $(m'_1, \ldots, m'_n)$ be two admissible topological orders. Since the graph is acyclic and semantics are path-independent under admissible reorderings, every adjacent swap of incomparable messages preserves the resulting state root. Any two topological sorts are related by a finite sequence of such swaps. Therefore the terminal state root is invariant. □

*Remark* 5.5. The path-independence assumption is not automatic. It must be designed into the message semantics or enforced by stricter ordering rules. This is one of the main engineering burdens of any real implementation.

### 5.2. **Knots.**

**Definition 5.6** (Knot)**.** A *knot* is a tuple

$$K = (\beta^\star, \rho^\star, \chi^\star, \eta^\star)$$

where $\beta^\star$ is a selected replayable braid, $\rho^\star$ is its terminal state root, $\chi^\star$ is an aggregate execution trace commitment, and $\eta^\star$ is an optional Bitcoin anchor commitment.

**Definition 5.7** (Selection functional)**.** A *knot selection functional* is a map

$$\mathcal{K} : \mathfrak{B} \to \mathcal{P}(\mathfrak{B})$$

which, given a finite set $\mathfrak{B}$ of candidate braids, returns the subset of admissible braid winners under local rules.

**Definition 5.8** (Heaviest replayable braid rule)**.** A selection rule is *heaviest replayable* if it returns

$$\mathcal{K}(\mathfrak{B}) = \arg\max_{\beta \in \mathfrak{B}_{\mathrm{rep}}} W(\beta)$$

where $\mathfrak{B}_{\mathrm{rep}}$ is the subset of replayable braids.

**Proposition 5.9** (Existence of maximizer)**.** *If $\mathfrak{B}_{\mathrm{rep}}$ is finite and nonempty, and $W(\beta)$ is real-valued, then at least one heaviest replayable braid exists.*

*Proof.* A real-valued function on a finite nonempty set attains a maximum. □

**Proposition 5.10** (Selection stability under dominated extensions)**.** *Let $\beta^\star$ be uniquely heaviest replayable in $\mathfrak{B}$. If a new candidate braid $\tilde{\beta}$ enters with*

$$W(\tilde{\beta}) < W(\beta^\star),$$

*then $\beta^\star$ remains the unique winner.*

*Proof.* Immediate from the uniqueness of the maximum. □

## 6. Compression Into Bitcoin Settlement

### 6.1. **Anchors.**

**Definition 6.1** (Anchor commitment)**.** An anchor commitment is a function

$$\eta : \mathcal{R} \times \mathcal{T} \to \mathcal{B}$$

that maps a state root and trace commitment into a Bitcoin-valid settlement artifact.

**Definition 6.2** (Compression ratio)**.** If a braid $\beta$ contains $n$ messages with total serialized payload

$$L(\beta) = \sum_{m \in V_\beta} \ell(m),$$

and its anchor artifact has Bitcoin weight $W_{\mathrm{BTC}}(\eta^\star)$, then define the anchoring compression ratio

$$\Gamma(\beta) = \frac{L(\beta)}{W_{\mathrm{BTC}}(\eta^\star)}.$$

**Lemma 6.3** (Nontrivial compression is possible)**.** *If the anchor commitment depends only on fixed-size root and trace summaries, then there exist families of braids* $(\beta_n)$ *with* $|V_{\beta_n}| \to \infty$ *for which*

$$\Gamma(\beta_n) \to \infty.$$

*Proof.* If anchor size is bounded independently of the braid payload size while braid payload grows unbounded, the quotient diverges. □

*Remark* 6.4. This is the essential formal benefit of an overlay. Expressive throughput can grow much faster than settlement throughput, provided authenticated compression is maintained.

## 7. Two-Stage Markets

### 7.1. Relay-side market.

**Definition 7.1** (Relay utility)**.** Let $u_r(m)$ denote the expected relay-side utility of a message $m$, including visibility, propagation probability, ranking, and likelihood of inclusion into a candidate braid.

**Definition 7.2** (Relay-side price)**.** The relay-side price of a message is the minimal expected cost

$$P_r(m)$$

required for $m$ to attain a target relay utility threshold.

In weakwork-weighted systems, $P_r(m)$ may be paid in hash effort, burns, membership fees, or other admission resources.

### 7.2. Settlement-side market.

**Definition 7.3** (Settlement utility)**.** Let $u_s(K)$ denote the expected utility of anchoring a knot $K$ into Bitcoin, including finality, timestamping, dispute minimization, and settlement enforceability.

**Definition 7.4** (Settlement-side price)**.** The settlement-side price of a knot is

$$P_s(K) = \text{fee required for Bitcoin inclusion of } \eta^\star.$$

**Theorem 7.5** (Two-stage pricing decomposition)**.** *Suppose an overlay checkpoint* $K$ *derives value both from local prominence and from Bitcoin anchoring. Then its total economic burden decomposes into*

$$P(K) = P_r(\beta^\star) + P_s(K)$$

*up to externalities and coordination overhead.*

*Proof.* All costs incurred prior to anchoring belong to acquiring relay-side attention, ranking, and replay rights for messages constituting $\beta^\star$. All costs incurred at anchoring belong to constructing and confirming the Bitcoin-valid anchor artifact. Since anchoring does not retroactively produce the earlier relay-side competition, and relay-side competition does not substitute for base Bitcoin inclusion, the burden decomposes additively except for external coordination effects. □

*Remark* 7.6. This theorem is trivial algebraically but important conceptually. It states that once the overlay has a real pricing surface, Bitcoin blockspace is no longer the only paid bottleneck. Rather, it becomes the second bottleneck.

## 8. Mining-Market Implications

### 8.1. Weakwork and hash allocation.

**Definition 8.1** (Hash allocation vector)**.** Let a hash-producing agent control total hash budget $H_{\text{tot}}$. An allocation vector is

$$\mathbf{q} = (q_B, q_W)$$

with

$$q_B + q_W = H_{\text{tot}}, \qquad q_B, q_W \geq 0$$

where $q_B$ is directed toward Bitcoin mining and $q_W$ toward overlay weakwork production.

**Definition 8.2** (Revenue functions)**.** Let

$$R_B(q_B)$$

denote expected Bitcoin mining revenue and

$$R_W(q_W)$$

denote expected overlay weakwork revenue or utility realization.

**Proposition 8.3** (Interior allocation condition)**.** *If $R_B$ and $R_W$ are differentiable, strictly concave, and the agent maximizes*

$$R_B(q_B) + R_W(q_W)$$

*subject to $q_B + q_W = H_{\text{tot}}$, then any interior optimum satisfies*

$$R'_B(q_B^\star) = R'_W(q_W^\star).$$

*Proof.* This is the first-order condition for constrained maximization under the resource constraint. □

*Remark* 8.4. Hence weakwork markets can compete with direct mining for hardware allocation even without changing Bitcoin consensus. The competition occurs in expected revenue space, not in validity space.

### 8.2. Anchor-driven miner incentives.

**Definition 8.5** (Anchor demand flow)**.** Let $\Lambda(t)$ be the stochastic process of demand for Bitcoin anchoring from overlay knots over time.

**Proposition 8.6** (Positive fee spillover)**.** *If knot anchors are represented as ordinary Bitcoin transactions paying fees, and if $\Lambda(t)$ is positive on a non-null set of times, then overlay anchoring creates positive fee spillover into Bitcoin mining revenue.*

*Proof.* Every anchor transaction that clears into a Bitcoin block contributes non-negative fee revenue to the miner of that block. Positive demand with positive accepted fee bids therefore adds expected revenue. □

**Corollary 8.7** (Subordinate complementarity)**.** *An overlay can simultaneously compete with mining for some off-chain hash effort and complement Bitcoin mining through on-chain anchor fee demand.*

*Proof.* Competition arises through $R_W(q_W)$ in the allocation problem, while complementarity arises through added miner fee flow from anchor inclusion. □

*Remark* 8.8. This mixed relation is crucial. It implies that a Bitcoin-rooted overlay need not be characterized simply as parasitic or purely symbiotic. It may be both competitive and complementary along different margins.

### 8.3. **Braid markets and mempool analogues.**

**Definition 8.9** (Braid inclusion premium)**.** Let $m$ be a message and $\beta^\star$ a candidate winning braid. Define the braid inclusion premium

$$\Pi_\beta(m) = \inf\{\Delta \geq 0 : m \text{ can be added to some winning candidate after increasing its priced resources}$$

*Remark* 8.10. This is the overlay analogue of the feerate premium required for likely mempool-to-block inclusion under congestion.

**Proposition 8.11** (Congestion premium)**.** *Suppose candidate winners are selected by heaviest replayable braid under a nonnegative additive weight model. Then as total candidate weight near the winning boundary increases, the braid inclusion premium for marginal messages is weakly increasing.*

*Proof.* Under the heaviest replayable criterion, a marginal message must contribute enough additional weight, directly or through package effects, to overcome the prevailing winning threshold. As that threshold rises, the required incremental contribution cannot fall absent structural changes in admissibility. Hence the premium is weakly increasing. □

## 9. Adversarial Considerations

**Definition 9.1** (Low-cost clutter attack)**.** A low-cost clutter attack is a strategy that attempts to saturate relay or braid attention using many messages whose production cost is below the social cost they impose on validation, storage, or ranking.

**Lemma 9.2** (Bounded-execution defense)**.** *If every admitted message must satisfy both weakwork admissibility and bounded deterministic replay cost, then the attack surface for low-cost clutter is reduced to the gap between marginal production cost and marginal verification cost.*

*Proof.* Weakwork imposes production-side cost, while bounded replay limits validation-side exposure. Any remaining attack must exploit a discrepancy between these two margins rather than unbounded computation or costless message emission. □

**Definition 9.3** (Weight inflation attack)**.** A weight inflation attack is an attempt to produce locally high-weight messages that are globally unreplayable or strategically conflictual, thereby distorting candidate braid comparisons.

**Proposition 9.4** (Replay filter necessity)**.** *In any heaviest-braid selection rule, replayability filtering is necessary to prevent globally invalid but locally weighty braids from dominating candidate order.*

*Proof.* Without replayability filtering, a candidate with arbitrarily high additive message weights could win despite failing deterministic execution, producing no coherent successor state. This contradicts the purpose of checkpoint selection. Therefore replayability must be part of the admissibility predicate before weight maximization. □

## 10. Client-Type Implications

**Definition 10.1** (Relay client)**.** A relay client verifies admission predicates and forwards messages according to local policy.

**Definition 10.2** (Execution client)**.** An execution client verifies relay admissibility, reconstructs candidate braids, and replays them into state roots.

**Definition 10.3** (Anchor client)**.** An anchor client verifies or constructs Bitcoin anchor artifacts corresponding to knots.

**Definition 10.4** (Integrated market client)**.** An integrated market client optimizes across relay pricing, weakwork generation, execution bandwidth, and Bitcoin anchoring.

**Proposition 10.5** (Client specialization pressure)**.** *If relay verification, execution replay, archival storage, and anchor construction have materially different cost structures, then market pressure favors specialization into distinct client types.*

*Proof.* When tasks differ substantially in bandwidth, latency, trust, and computational profiles, specialization lowers cost relative to universal replication by all participants. Standard comparative-advantage reasoning applies. □

*Remark* 10.6. This does not imply centralization by itself. It implies functional differentiation. Whether differentiation ossifies into centralization depends on switching costs, proof systems, data availability, and market openness.

## 11. Main Synthesis

We now gather the main logical arc.

**Theorem 11.1** (Pre-settlement market theorem)**.** *Assume the following:*

*(i) overlay messages require scarce admission resources,*
*(ii) admitted messages form acyclic dependency graphs,*
*(iii) candidate histories are weighted by a nonnegative additive rule,*
*(iv) only replayable candidates are eligible for checkpoint selection,*
*(v) checkpoints may be compressed into Bitcoin-valid anchors.*

*Then the overlay induces a pre-settlement market whose canonical objects are weighted replayable braids, whose clearing events are knots, and whose Bitcoin interface is a compressed settlement demand process.*

*Proof.* By (i), admission and ranking are scarce, hence economically priced. By (ii) and (iii), admitted messages may be assembled into finite weighted candidate histories. By (iv), only candidates admitting deterministic replay can compete for canonical checkpoint status, yielding knot-like clearing events. By (v), the resulting checkpoint can be projected into Bitcoin settlement through a valid anchor artifact. Therefore the overlay produces a distinct but subordinate market prior to Bitcoin settlement. □

**Corollary 11.2** (Mining implication)**.** *Under the hypotheses of the theorem, Bitcoin miners may be affected in two directions: indirectly through hardware competition with weakwork production, and directly through fee demand from anchor transactions.*

*Proof.* The first effect follows from hash allocation across weakwork and mining tasks. The second follows from anchor artifacts paying Bitcoin fees. □

**Conclusion 11.3.** A Bitcoin-adjacent overlay with weakwork and braid selection is best understood not as a rival consensus replacing Bitcoin, but as a market for scarce coordination preceding Bitcoin settlement. Its main novelty lies in assigning price to attention, order, and replay before assigning price to final block inclusion.

## 12. Conclusion

The strict mathematical picture is now visible. Weakwork gives a locally verifiable cost signal for admission. Braids transform admitted messages into weighted candidate histories rather than a mere bag of visible objects. Replayability forces these candidate histories to respect deterministic state transition rather than rhetorical prominence alone. Knots clear the competition by selecting a replayable winner. Anchors compress the winner's authenticated consequences into Bitcoin settlement artifacts. The entire structure therefore forms a two-stage market: first a market in overlay inclusion and ranked coordination, then a market in Bitcoin settlement.

This has several implications. First, Bitcoin blockspace remains the final thermodynamic sink rather than the sole venue for digital action. Second, mining is no longer economically isolated from higher-layer coordination once such coordination becomes priced. Third, the key design problem is not merely producing weight, but producing weight that survives replay, conflict resolution, and checkpoint selection. This makes the braid, not the isolated message, the central economic object of the overlay.

## 13. The Order of Agents in Kapnet

13.1. **Preliminaries.** We formalize Kapnet as a layered economy of scarce coordination rooted in Bitcoin settlement. The principal claim of this section is that Kapnet is not merely a set of users interacting over messages, but an ordered system of agents occupying distinct positions relative to physical work, temporal capital, coordination demand, treasury continuity, utility provision, and settlement.

**Definition 13.1** (Kapnet agent universe)**.** Let

$$\mathcal{A} = \{\mathcal{A}_0, \mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3, \mathcal{A}_4, \mathcal{A}_5, \mathcal{A}_6\}$$

denote the ordered family of Kapnet agent classes, where:

$$\begin{aligned}
\mathcal{A}_0 &:= \text{Hashproducers},\\
\mathcal{A}_1 &:= \text{Weakwork allocators and markets},\\
\mathcal{A}_2 &:= \text{Patient capital and hedlbit providers},\\
\mathcal{A}_3 &:= \text{Kapnetizens},\\
\mathcal{A}_4 &:= \text{DAO operators and treasury stewards},\\
\mathcal{A}_5 &:= \text{Utility providers},\\
\mathcal{A}_6 &:= \text{Settlement and anchor agents}.
\end{aligned}$$

*Remark* 13.2. The ordering above is not a caste ontology but a productive one. It is an ordering by principal economic function within Kapnet's circulation. An individual or organization may occupy several classes simultaneously.

13.2. **Agent classes.**

**Definition 13.3** (Hashproducer)**.** A *hashproducer* is an agent

$$a \in \mathcal{A}_0$$

who deploys hashproductive capital and converts physical inputs into computational output. Its productive state over an interval $t$ is modeled as

$$\mathsf{HP}_a(t) \in \mathbb{R}_{\geq 0},$$

where $\mathsf{HP}_a(t)$ denotes expected hash output over $t$.

**Definition 13.4** (Weakwork allocator)**.** A *weakwork allocator* is an agent

$$a \in \mathcal{A}_1$$

who transforms raw hashproductive output into priced admission rights, work packages, auctions, subscriptions, or standardized claims on future work.

**Definition 13.5** (HP-backed service credit)**.** An *HP-backed service credit*, abbreviated $\mathsf{HP\text{-}SBC}$, is a transferable or redeemable claim on future, reserved, or presently routable hashproductive service capacity.

**Definition 13.6** (Patient capital provider)**.** A *patient capital provider* is an agent

$$a \in \mathcal{A}_2$$

who contributes long-duration Bitcoin reserves and supplies hedlbit-denominated participation resources, reserve continuity, postage, or treasury stabilization.

**Definition 13.7** (Kapnetizen)**.** A *Kapnetizen* is an agent

$$a \in \mathcal{A}_3$$

who demands coordination rights in order to perform Kapnet activities, including posting, validating, proposing, voting, scripting, routing, or interacting with rooms and guilds.

**Definition 13.8** (DAO operator)**.** A *DAO operator* is an agent

$$a \in \mathcal{A}_4$$

who maintains institutional continuity by managing treasury balances, funding operations, disbursing rewards, topping up utility accounts, and preserving operational solvency of one or more collective entities.

**Definition 13.9** (Utility provider)**.** A *utility provider* is an agent

$$a \in \mathcal{A}_5$$

who delivers one or more operational services such as relay, storage, indexing, execution, moderation, delivery, or archival retention.

**Definition 13.10** (Settlement agent)**.** A *settlement agent* is an agent

$$a \in \mathcal{A}_6$$

who constructs, batches, verifies, or submits Bitcoin-valid anchor artifacts corresponding to Kapnet checkpoints, bundles, or compressed state commitments.

13.3. **The productive order.**

**Definition 13.11** (Agent order)**.** The *productive order* of Kapnet is the directed sequence

$$\mathcal{A}_0 \to \mathcal{A}_1 \to \mathcal{A}_3 \to \mathcal{A}_4 \to \mathcal{A}_5 \to \mathcal{A}_6,$$

supplemented by the temporal stabilization channel

$$\mathcal{A}_2 \to \mathcal{A}_4.$$

*Remark* 13.12. The first chain describes the principal route from physical work to Bitcoin anchoring. The second describes the route by which patient savings stabilize organizational duration.

**Theorem 13.13** (Kapnet circulation theorem)**.** *Assume:*

*(1) hashproductive capital can be converted into weakwork or related admission objects;*
*(2) Kapnet activities require scarce admission inputs;*
*(3) DAOs require operating balances to maintain continuity;*
*(4) utility providers require compensation to sustain service;*
*(5) selected outcomes may be compressed into Bitcoin-valid anchors.*

*Then Kapnet admits a nontrivial circulation economy in which physical work, temporal reserves, coordination demand, institutional budgets, and settlement capacity are linked by directed inter-agent flows.*

*Proof.* By (1), $\mathcal{A}_0$ can furnish productive scarcity to $\mathcal{A}_1$. By (2), $\mathcal{A}_1$ can sell or allocate admission resources to $\mathcal{A}_3$ and $\mathcal{A}_4$. By (3), $\mathcal{A}_4$ must maintain treasury continuity for collective action. By (4), $\mathcal{A}_4$ can fund $\mathcal{A}_5$ to obtain concrete services. By (5), outcomes produced through these services may be compressed into Bitcoin-valid artifacts by $\mathcal{A}_6$. Finally, $\mathcal{A}_2$ introduces reserve duration and postage support, ensuring that organizational continuity is not reducible to immediate transactional demand. Thus all listed flows exist and jointly induce a circulation economy. □

13.4. **Flow maps.**

**Definition 13.14** (Hash flow)**.** The *hash flow* is the map

$$F_H : \mathcal{A}_0 \to \mathcal{A}_1$$

sending raw productive work into priced weakwork, auctions, or HP-backed service instruments.

**Definition 13.15** (Weakwork flow)**.** The *weakwork flow* is the map

$$F_W : \mathcal{A}_1 \to \mathcal{A}_3 \cup \mathcal{A}_4$$

sending admission rights into end-user and institutional coordination demand.

**Definition 13.16** (Hedlbit postage flow)**.** The *hedlbit postage flow* is the map

$$F_{HB} : \mathcal{A}_2 \to \mathcal{A}_4$$

sending time-backed participation resources and postage liquidity into collective treasuries.

**Definition 13.17** (Operating budget flow)**.** The *operating budget flow* is the map

$$F_O : \mathcal{A}_4 \to \mathcal{A}_5$$

sending funded demand into relay, storage, execution, and related service layers.

**Definition 13.18** (Settlement flow)**.** The *settlement flow* is the map

$$F_S : \mathcal{A}_5 \to \mathcal{A}_6$$

sending selected or compressed outputs into the anchor construction and settlement interface.

**Proposition 13.19** (Directed scarcity transformation)**.** *Each principal flow*

$$F_H,\ F_W,\ F_{HB},\ F_O,\ F_S$$

*transforms one type of scarce resource into another without eliminating scarcity itself.*

*Proof.* Hash flow transforms physical work into weakwork or service claims. Weakwork flow transforms work-derived rights into coordination access. Hedlbit flow transforms time-backed reserves into postage or treasury endurance. Operating budget flow transforms balances into services. Settlement flow transforms service-side outputs into Bitcoin-valid settlement artifacts. Each stage changes the form of scarcity while preserving a cost boundary. □

13.5. **Treasury and continuity.**

**Definition 13.20** (DAO treasury state)**.** Let $D$ be a DAO. Its treasury state is the tuple

$$\mathcal{T}_D = (B_D, H_D, W_D, U_D, R_D),$$

where:

$$\begin{aligned} B_D &:= \text{base BTC reserves}, \\ H_D &:= \text{hedlbit-linked reserves}, \\ W_D &:= \text{weakwork reserves or entitlements}, \\ U_D &:= \text{utility balances across service providers}, \\ R_D &:= \text{outstanding reward obligations}. \end{aligned}$$

**Definition 13.21** (Treasury solvency)**.** A DAO treasury is *operationally solvent* over a horizon $[t_0, t_1]$ if it can meet all required obligations for postage, rewards, execution, storage, relay, and settlement across that interval.

**Lemma 13.22** (Utility continuity condition)**.** *If a DAO treasury is not operationally solvent over a horizon $[t_0, t_1]$, then either utility balances fall below service thresholds or reward obligations go unmet during that horizon.*

*Proof.* Operational solvency was defined as the ability to meet all required obligations across the horizon. Its failure implies that at least one such obligation cannot be met. □

**Proposition 13.23** (Need for treasury topping-up)**.** *Suppose a DAO consumes utility at rate vector*

$$\mathbf{u}(t) = \big(u_{\text{relay}}(t), u_{\text{store}}(t), u_{\text{exec}}(t), u_{\text{index}}(t)\big)$$

*and maintains balances*

$$\mathbf{b}(t) = \big(b_{\text{relay}}(t), b_{\text{store}}(t), b_{\text{exec}}(t), b_{\text{index}}(t)\big).$$

*If inflows fail to replenish balances faster than net consumption over sufficiently long intervals, then some service balance eventually reaches zero.*

*Proof.* Each balance follows a stock-flow relation of the form

$$b_i(t + \Delta t) = b_i(t) + \text{inflow}_i(\Delta t) - \text{consumption}_i(\Delta t).$$

If cumulative inflows remain below cumulative net consumption indefinitely, the corresponding balance decreases until exhaustion. □

*Remark* 13.24. This is the formal basis for the DAO-ops layer. Kapnet does not consist merely of message creation but of the maintenance of balances that make message processing durable.

## 14. Economy of Units

14.1. **Overview.** Kapnet inherits some units from Bitcoin and introduces additional derived units for participation, production, duration, and service pricing. This section defines a formal unit economy linking physical, monetary, temporal, and coordination quantities.

**Definition 14.1** (Unit family)**.** Let

$$\mathfrak{U} = \{\text{sat}, \text{B}, \text{WU}, \text{H/s}, \text{J}, \text{W}, \text{HB}, \text{HY}, \text{ASD}, \text{WW}\}$$

denote the principal family of units, interpreted respectively as satoshis, bytes, weight units, hash rate, joules, watts, hedlbits, hodlyield units, adjusted satdepth, and weakwork units.

### 14.2. **Bitcoin-native units.**

**Definition 14.2** (Satoshi)**.** The base monetary unit is the satoshi,

$$1 \text{ sat} = 10^{-8} \text{ BTC}.$$

**Definition 14.3** (Byte and weight unit)**.** If $x$ is a serialized transaction or message object, let

$$|x|_{\mathrm{B}}$$

denote its size in bytes. For Bitcoin settlement objects, let

$$|x|_{\mathrm{WU}}$$

denote its weight in weight units.

**Definition 14.4** (Fee density)**.** For a Bitcoin-valid settlement artifact $x$ with fee $f(x)$ in satoshis, define its fee density by

$$D_f(x) = \frac{f(x)}{|x|_{\mathrm{vB}}},$$

measured in sats per virtual byte.

### 14.3. **Physical production units.**

**Definition 14.5** (Hashrate)**.** For an agent $a$, its instantaneous hashrate is

$$h_a(t) \in \mathbb{R}_{\geq 0},$$

measured in hashes per second, abbreviated H/s.

**Definition 14.6** (Energy and power)**.** Let

$$E \in \mathbb{R}_{\geq 0}$$

be energy measured in joules (J), and let

$$P = \frac{dE}{dt}$$

be power measured in watts (W = J/s).

**Definition 14.7** (Hash-efficiency)**.** The *hash-efficiency* of a machine is

$$\eta_h = \frac{\mathrm{H/s}}{\mathrm{W}},$$

and its reciprocal energy intensity is

$$\iota_h = \frac{\mathrm{J}}{\mathrm{hash}}.$$

**Definition 14.8** (Electricity cost function)**.** If electricity price is $\pi_e$ denominated in satoshis per joule, then the electricity cost of running machine $a$ over interval $[t_0, t_1]$ is

$$C_e(a; [t_0, t_1]) = \pi_e \int_{t_0}^{t_1} P_a(t)\, dt.$$

*Remark* 14.9. One may equivalently parametrize electricity in fiat per kilowatt-hour and then map into satoshis through an exchange-rate variable. The satoshi-denominated form is preferable for intra-system economic comparison.

**Definition 14.10** (Heat output)**.** Assuming nearly all consumed electrical energy dissipates as heat, define heat output

$$Q_a([t_0, t_1]) \approx \int_{t_0}^{t_1} P_a(t)\, dt$$

measured in joules.

**Definition 14.11** (Heat per joule)**.** The quantity *heat per joule* is the ratio

$$\frac{Q}{E}.$$

Under ordinary ASIC operation this ratio is approximately 1 after thermal dissipation, since nearly all electrical input becomes heat.

*Remark* 14.12. The heat-per-joule ratio is formally trivial but economically relevant. When waste heat is captured or sold, the effective electricity cost of hashrate can be netted against a thermal recovery term.

**Definition 14.13** (Net energy cost)**.** If thermal recovery value is $\pi_q$ sats per joule of useful recovered heat, define net energy cost by

$$C_e^{\text{net}}(a; [t_0, t_1]) = (\pi_e - \pi_q) \int_{t_0}^{t_1} P_a(t)\, dt,$$

where $\pi_q = 0$ in the absence of recovery value.

14.4. **Weakwork units.**

**Definition 14.14** (Weakwork unit)**.** A *weakwork unit*, abbreviated WW, is a normalized measure of overlay proof-of-effort produced or purchased for message admission, ranking, or execution priority.

**Definition 14.15** (Weakwork price)**.** If a market quotes weakwork at rate

$$\pi_{ww}$$

measured in sats per weakwork unit, then a weakwork purchase of quantity $w$ costs

$$C_{ww} = \pi_{ww}\, w.$$

**Definition 14.16** (Weakwork density per byte)**.** For a Kapnet object $m$ with size $|m|_{\mathrm{B}}$ and weakwork allocation $w(m)$, define

$$D_{ww}(m) = \frac{w(m)}{|m|_{\mathrm{B}}},$$

measured in weakwork per byte.

### 14.5. **Hedlbits and postage.**

**Definition 14.17** (Hedlbit)**.** A *hedlbit*, denoted HB, is a time-conditioned participation resource derived from patient Bitcoin holding and used for postage, endurance, reserve backing, or prioritized rights in Kapnet.

**Definition 14.18** (Held reserve function)**.** Let agent $a$ hold satoshi reserve

$$S_a(t) \in \mathbb{R}_{\geq 0}$$

through time. A generic hedlbit accumulation functional is any map

$$\mathsf{HB}_a(t) = \mathcal{H}[S_a](t)$$

that is monotone nondecreasing in both reserve magnitude and reserve duration, subject to protocol-specific decay or eligibility rules.

**Definition 14.19** (Hedlbit per byte)**.** If postage for object $m$ is priced in hedlbits, define

$$D_{HB}(m) = \frac{\mathsf{HB}(m)}{|m|_{\mathrm{B}}},$$

measured in hedlbits per byte.

**Definition 14.20** (Hedlbit postage rate)**.** If a provider quotes postage at

$$\pi_{HB}$$

measured in sats per hedlbit, then the postage cost of object $m$ is

$$C_{HB}(m) = \pi_{HB}\,\mathsf{HB}(m).$$

*Remark* 14.21. Hedlbit postage differs from weakwork expenditure. Weakwork prices effort or anti-spam scarcity, while hedlbits price reserve patience and long-duration participation rights.

### 14.6. **Satdepth and adjusted satdepth.**

**Definition 14.22** (Satdepth)**.** For a satoshi-denominated reserve position with amount $S$ held over interval of duration $\Delta t$, define *satdepth* by

$$\mathrm{SD} = S \cdot \Delta t.$$

**Definition 14.23** (Adjusted satdepth)**.** Let $\varphi$ be a protocol-specific adjustment functional accounting for epoch weighting, lock quality, liquidity penalty, script class, or other temporal preference factors. Then *adjusted satdepth*, abbreviated ASD, is

$$\mathrm{ASD} = \varphi(S, \Delta t, \text{state parameters}).$$

*Remark* 14.24. Adjusted satdepth generalizes plain coin-age style measures. It allows Kapnet to privilege some forms of patience, stability, or script discipline more than others.

### 14.7. **Hodlyield.**

**Definition 14.25** (Hodlyield)**.** A *hodlyield* unit, abbreviated HY, is a reward quantity allocated as a function of adjusted satdepth.

**Definition 14.26** (Hodlyield rate)**.** Let

$$r_{hy}$$

denote the hodlyield rate, measured in hodlyield per adjusted satdepth. Then an eligible position with adjusted satdepth ASD receives

$$\mathrm{HY} = r_{hy} \cdot \mathrm{ASD}$$

over the relevant accounting interval, subject to budget constraints, epoch emission rules, or distribution curves.

**Proposition 14.27** (Monotonicity of hodlyield)**.** *If $r_{hy} \geq 0$ and the adjusted satdepth functional $\varphi$ is monotone in reserve amount and holding duration, then hodlyield is monotone in reserve amount and holding duration.*

*Proof.* Under the assumptions, ASD is monotone in both reserve amount and duration, and multiplication by nonnegative $r_{hy}$ preserves monotonicity. □

### 14.8. **Cross-unit pricing identities.**

**Definition 14.28** (Kapactivity cost decomposition)**.** Let $k$ be a Kapnet activity. Its total activity cost is

$$C(k) = C_{ww}(k) + C_{HB}(k) + C_u(k) + C_s(k),$$

where:

$$\begin{aligned} C_{ww}(k) &:= \text{weakwork expenditure}, \\ C_{HB}(k) &:= \text{hedlbit postage expenditure}, \\ C_u(k) &:= \text{utility service expenditure}, \\ C_s(k) &:= \text{Bitcoin settlement expenditure, if any.} \end{aligned}$$

**Definition 14.29** (Utility expenditure)**.** If a utility provider charges price vector

$$\Pi_u = (\pi_{\text{relay}}, \pi_{\text{store}}, \pi_{\text{exec}}, \pi_{\text{index}})$$

and activity $k$ consumes service vector

$$\mathbf{s}(k) = (s_{\text{relay}}, s_{\text{store}}, s_{\text{exec}}, s_{\text{index}}),$$

then utility expenditure is

$$C_u(k) = \langle \Pi_u, \mathbf{s}(k) \rangle.$$

**Definition 14.30** (Anchor cost)**.** If activity $k$ or a batch containing it is anchored into Bitcoin via settlement artifact $x$, define

$$C_s(k) = \alpha_k \, f(x),$$

where $\alpha_k$ is the accounting share of $k$ in the anchor fee allocation.

**Theorem 14.31** (Multi-unit activity theorem)**.** *Every nontrivial Kapactivity requiring admission, persistence, and optional settlement can be priced as a composition of at least one coordination unit, at least one service unit, and optionally one settlement unit.*

*Proof.* Admission requires either weakwork, hedlbit postage, or another coordination right. Persistence requires relay, storage, execution, or related service expenditure. If the activity is anchored into Bitcoin, a settlement fee component also appears. Thus the total cost decomposes into coordination, service, and optional settlement terms. □

## 14.9. **Unit conversion and market surfaces.**

**Definition 14.32** (Unit conversion surface)**.** A *unit conversion surface* is a family of market maps converting between Kapnet-relevant units, for example:

$$\begin{aligned}
&\Phi_1 : \mathrm{J} \leftrightarrow \mathrm{sat},\\
&\Phi_2 : \mathrm{H/s} \leftrightarrow \mathrm{WW},\\
&\Phi_3 : \mathrm{HB} \leftrightarrow \mathrm{sat},\\
&\Phi_4 : \mathrm{ASD} \leftrightarrow \mathrm{HY},\\
&\Phi_5 : \mathrm{B} \leftrightarrow \mathrm{HB},\\
&\Phi_6 : \mathrm{WU} \leftrightarrow \mathrm{sat}.
\end{aligned}$$

*Remark* 14.33. Kapnet's economy is not governed by a single numeraire in practice, even if satoshis are used as the preferred unit of final accounting. Several economically meaningful exchanges occur first in energy, heat, work, patience, storage, or service space.

**Proposition 14.34** (Non-reducibility of unit strata)**.** *Suppose Kapnet distinguishes between weakwork scarcity, hedlbit scarcity, utility scarcity, and Bitcoin settlement scarcity. Then no single unit fully captures all resource constraints unless the protocol collapses these distinctions by policy.*

*Proof.* Weakwork, hedlbits, service balances, and Bitcoin fees correspond to distinct bottlenecks with distinct production and replenishment mechanisms. A single scalar numeraire may price them ex post, but does not erase their structural non-equivalence. □

## 14.10. **A concise unit summary.**

**Summary 14.35.** The principal unit interpretations are:

| | |
|---|---|
| sat | monetary settlement unit, |
| B, WU | message and settlement size units, |
| H/s | productive hash rate unit, |
| J, W | energy and power units, |
| WW | work-derived admission unit, |
| HB | patience-derived postage and reserve unit, |
| SD, ASD | time-weighted reserve depth units, |
| HY | reward unit derived from adjusted reserve depth. |

14.11. **Interpretive conclusion.**

**Conclusion 14.36.** The economy of Kapnet is a multi-unit economy. Satoshis price final monetary settlement. Hashrate and joules price physical production. Weakwork prices admission and ranking effort. Hedlbits price patience and postage. Adjusted satdepth measures temporal reserve quality. Hodlyield rewards such reserve quality. Utility balances price the continuity of actual services. The order of agents in Kapnet is therefore inseparable from the order of units they primarily produce, trade, consume, or govern.

**Acknowledgment.** Special thanks to @itme_brain and @mark_hodl for fueling the contrarian spirit.

## References

1. Nakamoto, S. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System*. https://bitcoin.org/bitcoin.pdf
2. Bitcoin Developer Documentation. *Transactions*. https://developer.bitcoin.org/devguide/transactions.html
3. Walker, G. et al. *UTXO — Unspent Transaction Output*. Learn Me A Bitcoin. https://learnmeabitcoin.com/technical/transaction/utxo/
4. Walker, G. et al. *Mempool — Bitcoin's Waiting Room For Transactions*. Learn Me A Bitcoin. https://learnmeabitcoin.com/technical/mining/memory-pool/
5. Walker, G. et al. *Merkle Root — A Fingerprint for the Transactions in a Block*. Learn Me A Bitcoin. https://learnmeabitcoin.com/technical/block/merkle-root/
6. Walker, G. et al. *Mining*. Learn Me A Bitcoin. https://learnmeabitcoin.com/technical/mining/

[1] Founder and Headmaster, Central Bitcoin Timechain Creds
*Email address*: n0-space@bitcoincreds.com